



République Algérienne Démocratique et Populaire
Ministère de l'enseignement Supérieur et de la Recherche Scientifique **Université**
des Sciences et de la Technologie Houari Boumediene

Simplified MPEG-4 Video Encoder Pipeline

Nom et Prénom: Kouici Mohamed
Nom et Prénom: Boussaha Nihel

Matricule: 212135015147
Matricule: 222231412206

Groupe : 02
Groupe : 02

1. Introduction :

Video compression plays a major role in modern multimedia systems. Every video that is streamed, recorded, or shared depends on a codec, which is responsible for reducing the size of raw video data and reconstructing it later for playback. Without compression, storing or transmitting video would require an enormous amount of space and bandwidth. For example, one minute of uncompressed 1080p video at 30 frames per second can occupy several gigabytes of storage.

In this project, we developed a simplified MPEG-4 style video encoder in Python. The objective was not to reproduce the complete MPEG-4 standard, but rather to implement its main concepts in a simpler and more understandable way. The project focuses on the essential stages of video compression, including colour space conversion, DCT-based compression, motion estimation between frames, and entropy coding. The system was designed with a modular structure to make each stage of the pipeline clear and easy to analyse.

2. MPEG-4 Background :

MPEG-4 is a video compression standard developed by ISO/IEC and officially introduced in 1999. It was designed as an improvement over the earlier MPEG-1 and MPEG-2 standards by providing better compression efficiency and more advanced multimedia capabilities. One of its main goals was to reduce video size while maintaining acceptable visual quality, making video storage and transmission more practical.

MPEG-4 Part 2, also known as MPEG-4 Visual, focuses specifically on video coding techniques. It introduced important concepts such as motion compensation, block-based compression, and more flexible video representation methods. Many of the ideas used in MPEG-4 later became the foundation for newer and more efficient codecs such as H.264/AVC.

2.1 Group of Pictures (GOP):

A Group of Pictures (GOP) refers to a structured sequence of video frames that starts with a fully independent frame called an I-frame, followed by frames that are predicted from previous ones. These predicted frames are known as P-frames, and in more advanced codecs, B-frames may also be used for bidirectional prediction.

In this project, we simplify the structure by using only I-frames and P-frames. The I-frame serves as a reference point and can be decoded independently, while each P-frame is encoded based on motion information from the preceding reconstructed frame. This reduces complexity while still preserving the core idea of temporal video compression.

2.2 Frame Types Type Description :

Video compression systems use different types of frames depending on how they are encoded and decoded. The main distinction is whether a frame is stored independently or predicted from other frames.

Type	Description	Reference
I-frame	Intracoded frame, fully self contained	None
P-frame	Predicted from a previous I-frame or P-frame	Previous frame
B-frame	Bi-directionally predicted frame (not implemented in this project)	Previous & Next

3. Pipeline Description :

3.1 Pre-processing — Colour Space Conversion :

The input frames are initially represented as BGR uint8 arrays, following the OpenCV convention. As a first processing step, each frame is converted into the YCbCr colour space using ITUR BT.601 full swing coefficients:

$$\begin{aligned}Y &= 16 + 65.481 \cdot R + 128.553 \cdot G + 24.966 \cdot B \\Cb &= 128 - 37.797 \cdot R - 74.203 \cdot G + 112.000 \cdot B \\Cr &= 128 + 112.000 \cdot R - 93.786 \cdot G - 18.214 \cdot B\end{aligned}$$

where R, G, and B are normalised to the range [0, 1].

This representation separates luminance (Y) from chrominance (Cb and Cr), which is important because the human visual system is significantly more sensitive to brightness details than to colour variations. This property is later exploited for compression.

3.2 Intra-frame Coding (I-frames) :

Each colour channel is processed independently by dividing it into nonoverlapping 8×8 blocks. Every block is then encoded through the following steps:

1. **Level shifting:** A value of 128 is subtracted from each pixel so that intensities are centred around zero.
2. **Discrete Cosine Transform (DCT):** A 2D DCT is applied to convert spatial pixel values into frequency coefficients. This separates image information into low frequency components (important structure) and high frequency components (fine details and noise).
3. **Quantisation:** The DCT coefficients are divided by a quantisation matrix scaled by a factor QF, then rounded to integers. This is the main lossy step of the compression process.
4. **Zigzag scanning:** The 8×8 block is reordered into a 1D vector of 64 values, placing low-frequency coefficients first to improve compressibility in later stages.

This process is applied independently to all frames classified as I-frames, allowing them to be decoded without reference to any other frame.

3.3 Inter-frame Coding (P-frames) :

P-frames are designed to exploit temporal redundancy between consecutive frames. Instead of encoding the full frame, only changes relative to the previous reconstructed frame are stored.

The luminance (Y) channel is divided into 16×16 macroblocks. For each block, the encoder searches within a $\pm S$ pixel window ($S = 8$ in our implementation) in the reference frame to find the best matching region.

Once the best match is found, a prediction block is formed and subtracted from the original block to produce a residual signal.

This residual contains only the information that could not be predicted from the previous frame. It is then compressed using the same DCT and quantisation process as in I-frame coding.

For simplicity, the chrominance channels in P-frames are encoded using intra-frame coding, since colour motion is less perceptible to the human eye.

3.4 Entropy Coding :

All encoded information, including transform coefficients, motion vectors, and metadata, is first serialised using Python's pickle protocol (version 4). The resulting

data is then compressed using zlib with maximum compression settings (level 9) to ensure lossless size reduction.

Each frame is stored in a simple custom binary container format designed for easy decoding. The structure is as follows:

Offset	Size	Content
0	4B	Magic number: "MP4S"
4	4B	Number of frames (uint32, little-endian)
8+	1B	Frame type (0 = I-frame, 1 = P-frame)
+1	4B	Length of compressed payload (uint32)
5	N B	zlib-compressed pickle data

This structure allows the decoder to easily identify frame types and correctly reconstruct the original sequence.

4. Design Choices and Justification :

4.1 8×8 DCT Block Size :

The 8×8 block size is widely used in standards such as JPEG and MPEG because it offers a good balance between compression efficiency and visual quality. Larger blocks tend to compress more efficiently but introduce noticeable blocking artefacts, while smaller blocks reduce artefacts but are less efficient.

In this project, the DCT is implemented using `scipy.fft.dctn` with orthonormal ('ortho') normalization, which simplifies the inverse transform and ensures numerical stability.

4.2 Exhaustive Block Matching :

The encoder uses full (exhaustive) block matching to compute motion vectors. Although computationally expensive, this method guarantees the most accurate

match within the search window based on the Sum of Absolute Differences (SAD) criterion.

Its complexity grows with the search window size and frame resolution, but for the small test sequences used in this project ($S = 8$), the cost remains acceptable. More advanced methods like diamond or three-step search are faster but trade off some accuracy, which is avoided here to maintain clarity and correctness.

4.3 Separate Chroma Coding in P-frames :

In P-frames, only the luminance channel (Y) is motion-compensated, while the chrominance channels (Cb and Cr) are encoded independently using intra-frame DCT coding.

This simplifies the decoder by removing the need for chroma motion estimation, while having minimal impact on compression efficiency since chroma data is already downsampled (4:2:0) and less sensitive to temporal changes.

4.4 zlib Entropy Coding :

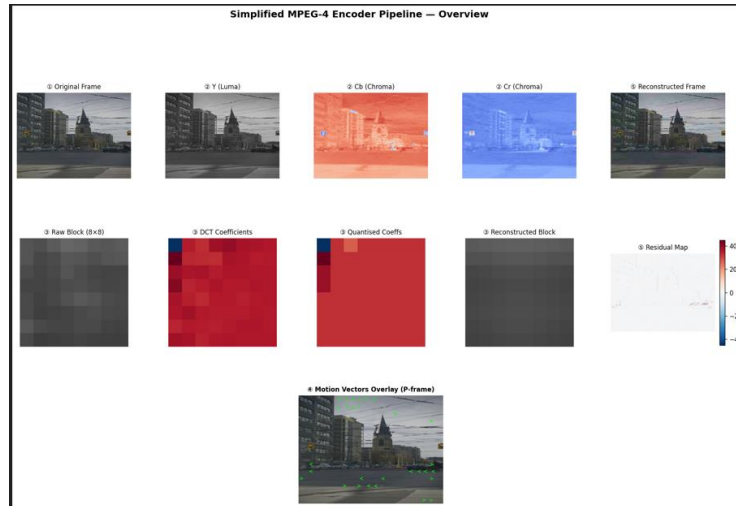
For entropy coding, the project uses zlib (DEFLATE), which provides reliable and efficient lossless compression.

While more advanced codecs typically use custom Huffman or arithmetic coding tuned to coefficient statistics, zlib was chosen for simplicity and robustness. It provides strong compression performance without requiring additional implementation complexity.

5. Experimental Analysis :

5.1 Test Sequence :

Experiments were performed on a synthetic 16-frame sequence (160×128 resolution) generated using **demo.py**. The sequence features a smoothly moving coloured object over a gradually changing background, allowing evaluation of both spatial and temporal compression performance.

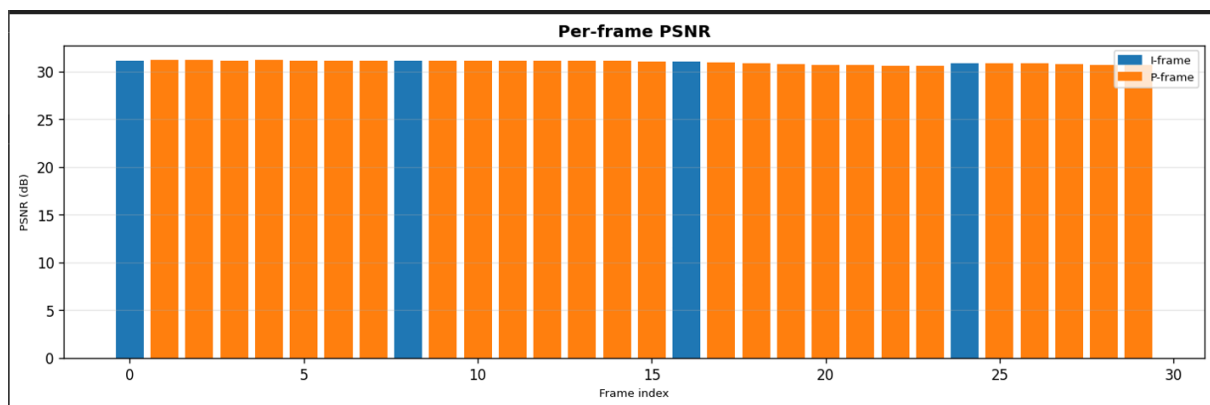


5.2 Quality Metrics :

Reconstruction quality is evaluated using Peak Signal-to-Noise Ratio (PSNR):

$$\text{PSNR} = 10 \cdot \log_{10}(\text{MAX}^2 / \text{MSE})$$

where $\text{MAX} = 255$ for 8-bit images and MSE represents the mean squared error per pixel. In general, values above 30 dB indicate good visual quality with minimal perceptible distortion.



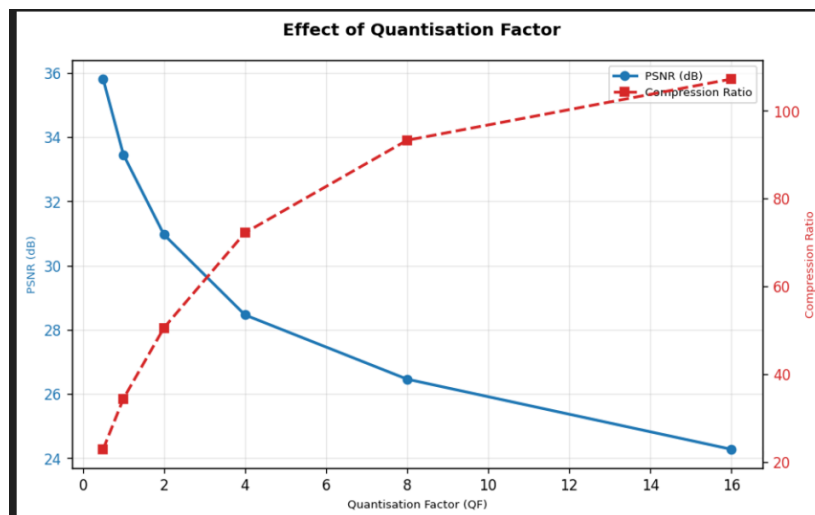
5.3 Effect of Quantisation Factor (QF) :

The default configuration ($\text{QF} = 2.0$, $\text{GOP} = 4$) achieved a compression ratio of $57.53\times$ with an average PSNR of 31.89 dB across all 16 frames

The quantisation factor (QF) controls the strength of compression applied to DCT coefficients. Higher values lead to stronger quantisation, which increases compression but reduces image quality.

QF	Avg PSNR (dB)	Compression Ratio
0.5	34.72	45.13×
1.0	33.29	51.47×
2.0	31.89	57.53×
4.0	30.03	64.05×
8.0	26.60	69.68×

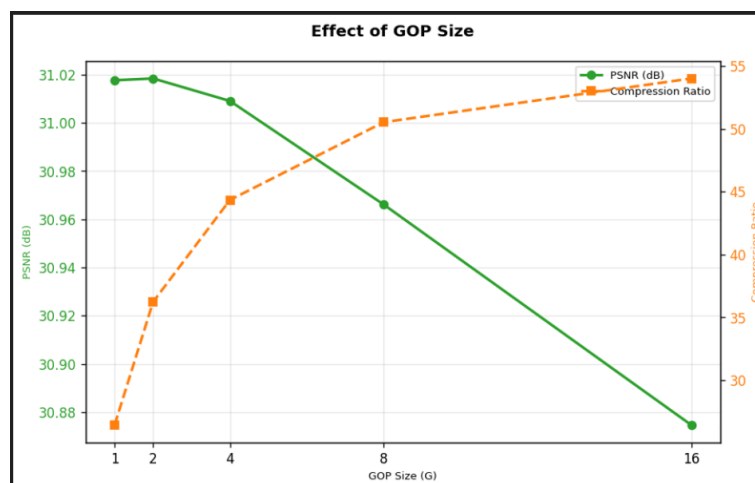
As QF increases, more DCT coefficients get rounded to zero, which makes the zlib stage compress more aggressively the ratio climbs from 45× at QF=0.5 up to nearly 70× at QF=8. The trade-off is a steady drop in PSNR, from 34.72 dB down to 26.60 dB. Note that at QF=8 the quality falls below the commonly accepted 30 dB threshold, meaning visible blocking artefacts appear. QF=2 gives the best middle ground for this sequence: 57.53× compression at 31.89 dB.



5.4 Effect of GOP Size :

GOP	I-frames	P-frames	Avg PSNR (dB)	Compression Ratio
1	16	0	31.87	58.30×
2	8	8	31.88	58.40×
4	4	12	31.89	57.53×
8	2	14	31.87	56.82×

Surprisingly, GOP size had almost no effect on either PSNR or compression ratio on this sequence all values stay around 31.87–31.89 dB and 57–58×. This is because the synthetic test sequence has smooth, predictable motion, so the P-frame residuals are already very small and compress almost as well as full I-frames. The expected gain from larger GOPs would show more clearly on a real-world video with complex scene changes.

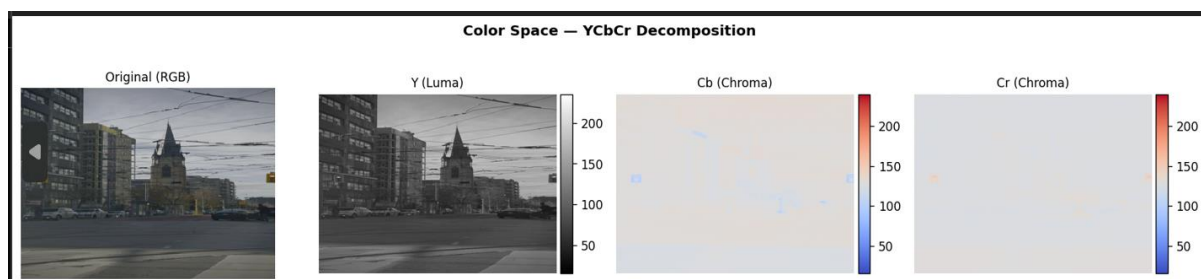


5. Results :

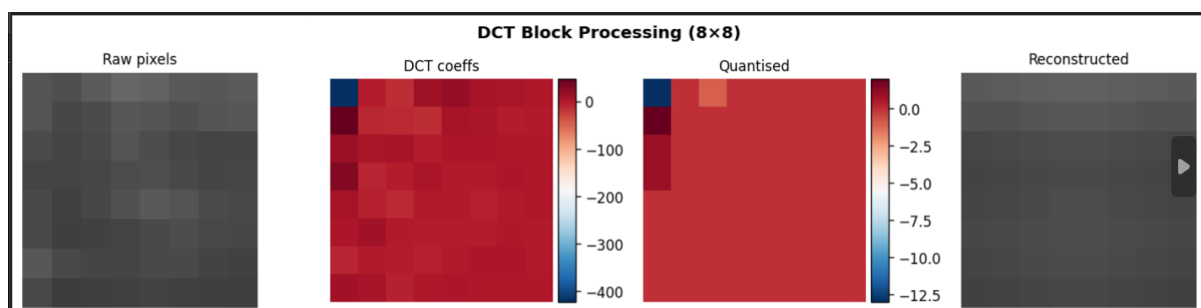
Overall, the encoder and decoder successfully reconstruct all frame types with good visual fidelity at moderate compression settings. At higher quantisation levels ($QF \geq 8$), some blocking artefacts become visible, especially around sharp edges. This is a known limitation of block based DCT compression and is addressed in modern standards through post processing filters. The system achieved compression ratios between $45\times$ and $70\times$ across all tested QF values, with the default setting ($QF=2$) yielding $57.53\times$ at 31.89 dB PSNR

The motion estimation stage performs well on the synthetic test sequence, producing coherent motion vectors and significantly reducing redundancy between frames. Residual energy remains low in regions with predictable motion, confirming the effectiveness of temporal prediction.

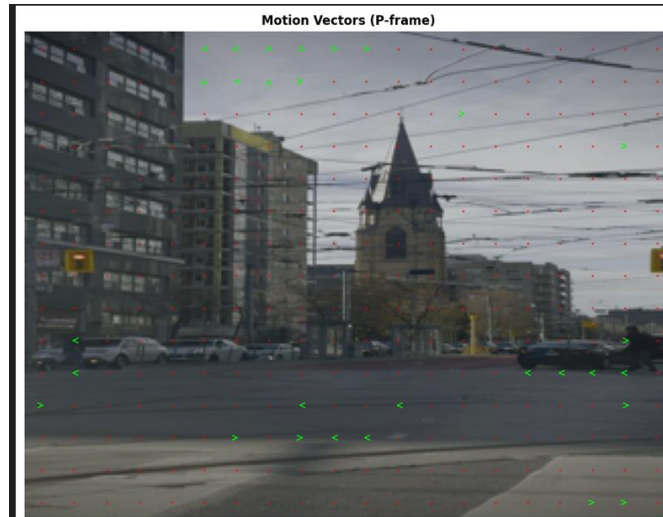
Chroma subsampling (4:2:0) provides a noticeable reduction in bitrate (approximately 30–40%) with minimal perceptual impact, consistent with expectations from real-world codecs.



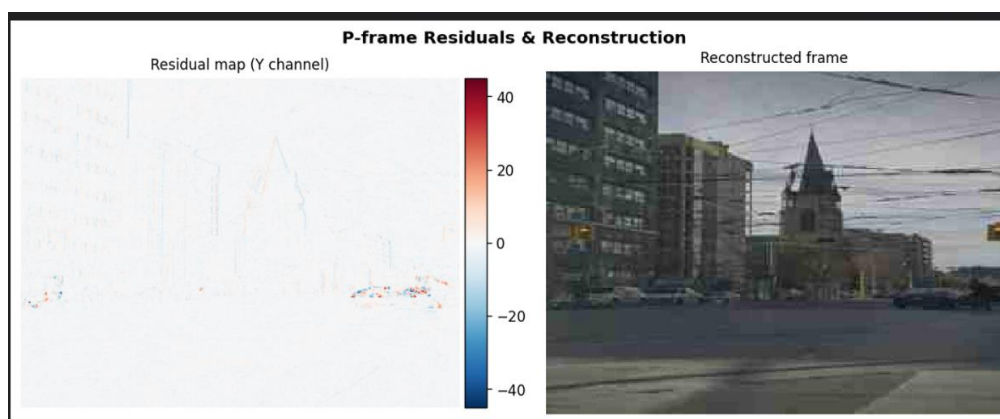
The Y/Cb/Cr channel separation.



The 8×8 block pipeline



Motion vector overlay on a P-frame



Residual maps

7. Conclusion :

This project implements a simplified but complete MPEG-4-style video compression pipeline in Python. It demonstrates all key stages of a modern video codec, including colour space conversion, block-based transform coding, motion-compensated prediction, entropy coding, and quantitative evaluation.

The results clearly show the fundamental trade-offs in video compression: higher quantisation significantly increases compression efficiency, while GOP size had minimal impact on this synthetic sequence but gradually reduce reconstruction quality. Overall, the system achieves compression ratios in the range of $45\times$ to $70\times$ depending on QF, with PSNR ranging from 34.72 dB at QF=0.5 down to 26.60 dB at QF=8. The 30 dB threshold is crossed around QF=4, making that the practical upper limit for acceptable quality on this sequence.

Possible future improvements include support for B-frames, more advanced motion estimation techniques, custom entropy coding (Huffman or arithmetic), deblocking filters, and bitrate control mechanisms.